Retrievable Gradients: Continual Post-Training Without Cumulative Weight Drift

Weihang Su[*1,2], **Jiacheng Kang**[1], **Jingyan Xu**[1], **Qingyao Ai**[†2,1], **Jianming Long**[1], **Hanwen Zhang**[1], **Bangde Du**[1], **Xinyuan Cao**[1], **Min Zhang**[1], **Yiqun Liu**[1]

[1]Department of Computer Science and Technology, Tsinghua University
[2]Quancheng Laboratory

Abstract

Continual post-training enables models to absorb emerging knowledge after deployment, but repeatedly updating shared parameters can accumulate weight drift, potentially causing catastrophic forgetting and degrading general capabilities. Retrieval-augmented generation avoids such parameter drift, yet often lacks the depth of parametric knowledge integration. In this paper, we propose REGRAD (Retrievable Gradients), a new paradigm that treats gradients as retrievable units of knowledge. REGRAD pre-computes document-specific gradients offline, stores them in an indexed Gradient Bank, and retrieves only query-relevant gradients at inference time for temporary weight adaptation. However, raw language-modeling gradients are optimized for token-level document reconstruction rather than for query-driven knowledge use. We therefore introduce a bi-level meta-learning objective that reshapes document-derived gradients into generalizable adaptation signals for downstream tasks. Experiments across general and domain-specific settings show that REGRAD outperforms CPT and RAG baselines, enabling scalable and reversible parametric knowledge injection without accumulating weight drift[1].

1 Introduction

A fundamental challenge in deploying large language models (LLMs) is the mismatch between their static parameters and the dynamic nature of real-world knowledge (Lazaridou et al., 2021; Petroni et al., 2019). As new information emerges after deployment, models require mechanisms that can update their knowledge without sacrificing their foundational capabilities (Jang et al., 2021). Continual post-training (CPT) has emerged as a prevalent approach for this purpose, as it enables models to internalize new knowledge directly into their parametric memory (Gururangan et al., 2020a). However, this strength is tied to an inherent limitation: CPT incorporates new knowledge by repeatedly updating the same shared parameters. As updates accumulate over time or across large-scale corpora, the model weights gradually drift away from their original parameter state, which can lead to catastrophic forgetting and degradation of general capabilities such as reasoning and instruction following (Xia et al., 2024; Lu et al., 2025; Lin et al., 2024; Cheng et al., 2023; Jindal et al., 2024; Wang et al., 2025d). Moreover, CPT accumulates document-induced parameter updates into a single globally shared parameter state. However, each downstream query typically requires only a small subset of the updated knowledge. Since the resulting parameters are shared across all queries, the knowledge relevant to a given query is inevitably mixed with many unrelated parameter modifications. Together, these limitations expose a central tension in knowledge updating: how can we obtain the benefits of parametric knowledge integration without continuously accumulating weight drift?

Retrieval-augmented generation (RAG) (Lewis et al., 2020) offers an alternative way to incorporate external knowledge by retrieving relevant textual knowledge at inference time rather than modifying model parameters. This design naturally avoids parameter drift and helps preserve the base model's general capabilities. Nevertheless, RAG leaves new knowledge outside the model's parameters and relies on in-context knowledge injection, where external knowledge is provided as part of the input context rather than incorporated into the model's parameters (Su et al., 2025e). This distinction is important because in-context and parametric knowledge integration operate through different computational mechanisms. In-context knowledge injection introduces new information only through

[*]Email: oneal2000@126.com
[†]Corresponding Author: aiqy@tsinghua.edu.cn
[1]We have open-sourced all the code, data, and models at: https://github.com/oneal2000/ReGrad

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 27407–27426
October 24-29, 2026 ©2026 Association for Computational Linguistics

the input context, where it influences the model through input-dependent inference-time computations rather than persistent parameter updates. In contrast, prior studies suggest that much of an LLM's world knowledge and reasoning capabilities are encoded in its parameters (Yu and Ananiadou, 2024; Nanda et al., 2023). Consequently, externally injected knowledge may not be utilized in the same way as knowledge internalized in the model's parameters (Su et al., 2025e; Tan et al., 2025). Thus, CPT and RAG present complementary strengths but also a clear trade-off: CPT provides parametric integration but accumulates irreversible updates, whereas RAG is reversible and selective but remains non-parametric.

In this work, we revisit the basic unit of parametric knowledge injection: the gradient. In standard training, gradients are treated as transient optimization signals: they are computed from training instances, applied to update the model weights, and then discarded (Rumelhart et al., 1986). Yet a gradient is more than an optimization signal: it also encodes how the model parameters should change in response to a particular document or training instance. This observation suggests a different view: rather than permanently accumulating the effects of all document-induced gradients in a single globally shared parameter state, we can store these gradients offline as reusable parametric adaptation units, retrieve a query-relevant subset at inference time, and use them to induce query-specific parameter adaptation. Such a mechanism retains the parametric nature of CPT by incorporating new information through parameter-space adaptation, while preventing updates from being persistently accumulated across unrelated documents or queries.

Based on this idea, we propose REGRAD (Retrievable Gradients), a new paradigm that treats gradients as retrievable units of knowledge. REGRAD first pre-computes document-specific gradients offline and stores them in an indexed Gradient Bank. At inference time, given a user query, REGRAD retrieves the most relevant gradients and temporarily applies them to the model weights for query-specific adaptation. After generation, the adapted weights are discarded, and the base model is restored. In this way, REGRAD decouples knowledge acquisition from permanent weight modification: knowledge can be injected into the model parameters when needed, while the shared base parameters remain unchanged across queries. Importantly, REGRAD can be used either as a standalone parameter-only adaptation method, where no retrieved text is placed in the input context, or together with in-context RAG, where retrieved gradients are applied to the model while retrieved documents are provided as textual evidence.

A remaining challenge is that raw gradients from standard language-modeling objectives are not necessarily suitable for downstream knowledge use. Next-token prediction encourages document reconstruction, whereas downstream tasks require query answering, implicit fact extraction, and reasoning over document content. As a result, simply caching language-modeling gradients may overfit surface-level token patterns rather than improve query-driven generalization. To address this problem, REGRAD introduces a bi-level meta-learning objective that reshapes document-derived gradients into generalizable adaptation signals. The inner update applies the document-induced gradient to the model, while the outer objective evaluates the adapted model on downstream supervision. Through this objective, REGRAD meta-learns gradient formation and application, so that temporary adaptation improves downstream behavior rather than merely reconstructing document text.

We conduct extensive experiments across both general and domain-specific settings, including law and medicine. In the parameter-only setting, REGRAD outperforms continual post-training and other parametric updating baselines, demonstrating that document-induced gradients can serve as effective, retrievable adaptation units. When combined with in-context knowledge injection, REGRAD further improves performance, suggesting that retrievable gradients and textual contexts provide complementary forms of knowledge injection. These results position REGRAD not as a replacement for RAG, but as a reversible parameter-side mechanism that can be used independently or integrated with existing retrieval-augmented systems. In summary, this paper makes the following contributions:

- We propose REGRAD, a new paradigm that stores gradients as retrievable units, enabling query-specific parametric updating without permanently modifying the model parameters.
- We introduce a bi-level meta-learning objective that reshapes document-derived language-modeling gradients for knowledge use rather than document reconstruction.
- We empirically show that REGRAD outperforms CPT and RAG and can be combined with in-context RAG to achieve complementary gains.

2 Related Work

REGRAD is related to three lines of work: continual post-training (CPT), retrieval-augmented generation (RAG), and test-time training. CPT injects new knowledge into model parameters, but repeated updates may cause catastrophic forgetting and capability drift, motivating mitigation strategies such as regularization, distillation, and replay (Kirkpatrick et al., 2017; Li and Hoiem, 2017; Lopez-Paz and Ranzato, 2017). Knowledge editing also modifies parameters, but aims for localized factual updates with limited side effects (Meng et al., 2022a,b; Mitchell et al., 2021, 2022). Unlike these methods, REGRAD retrieves temporary, query-relevant parameter shifts without permanently changing the base model. RAG instead conditions generation on retrieved evidence (Lewis et al., 2020; Guu et al., 2020; Borgeaud et al., 2022). The closest line is Parametric RAG, which converts documents into retrievable parameter modules (Su et al., 2025e). Finally, test-time training performs online gradient updates during inference (Hardt and Sun, 2024; Chen et al., 2023), whereas REGRAD avoids online backpropagation by retrieving precomputed gradients. Further review and discussion of related work is provided in Appendix A.

3 The ReGrad Framework

In this section, we introduce our proposed REGRAD framework, shown in Figure 1. We first present the overview of our entire framework in § 3.1. We then describe our proposed framework, which operates in a three-stage lifecycle: (1) Meta-learning to align unsupervised document gradients with downstream QA tasks (§ 3.2); (2) Gradient Bank Construction to store these aligned gradients offline (§ 3.3); and (3) Online Inference to perform retrieval-augmented parametric updates (§ 3.4). Unless otherwise specified, REGRAD denotes the parametric-only setting without retrieved passages in the QA prompt.

3.1 Overview

Let $\mathcal{M}_W$ denote a pre-trained LLM with base parameters W, and let $\mathcal{D} = \{d_i\}_{i=1}^N$ be an external knowledge corpus. Given a test-time query q, REGRAD aims to inject relevant knowledge from $\mathcal{D}$ through a temporary parametric update, while restoring the model to $\mathcal{M}_W$ after generation. The framework consists of three stages. First, a meta-learning stage learns adaptation-friendly parameters that turn document-induced gradients into QA-oriented updates. Second, these parameters are used to build a Gradient Bank by pre-computing document-specific gradients. Third, at inference time, query-relevant gradients are retrieved and aggregated to produce a temporary update in the parameter space.

3.2 Stage 1: Meta-learning for Initialization

In this stage, REGRAD follows a MAML-style bi-level optimization (Finn et al., 2017): an *inner* unsupervised update extracts knowledge from retrieved documents, and an *outer* QA loss evaluates whether the updated model can answer downstream questions without seeing the documents in the prompt. The goal of Stage 1 is to learn an initialization and step sizes such that an unsupervised language-modeling gradient computed from a document can be used as a generalizable one-step adaptation for downstream question answering. This stage is the key to making stored gradients usable beyond surface-form reconstruction: during deployment, we will only have access to the knowledge corpus documents (no QA labels), yet we want the retrieved gradient updates to consistently improve query-answering behavior. To make document-specific gradients storable and retrievable at the corpus scale, REGRAD avoids computing or storing gradients over the full model parameters W. Instead, we introduce a compact adaptation subspace by inserting LoRA adapters into selected transformer blocks and restricting adaptation to their parameters θ. The resulting model is $\mathcal{M}_{(W,\theta)}$, where W remains frozen, and all document-induced gradients are computed with respect to θ. This low-rank parameterization substantially reduces the storage cost of the Gradient Bank while retaining sufficient capacity to encode instance-level knowledge updates.

3.2.1 Inner objective.

Given a document (or chunk[2]) $C = \{c_t\}_{t=1}^T$, we compute its gradient representation by differentiating the standard language modeling objective:

$$\tilde{\mathcal{L}}_C(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_{(W,\theta)}(c_t \mid c_{<t}). \quad (1)$$

We denote the resulting gradient in the plug-in space as a knowledge atom, and apply a virtual update step:

[2]For our main experiment, we treat each d_i as a chunked passage of length up to 200 tokens.

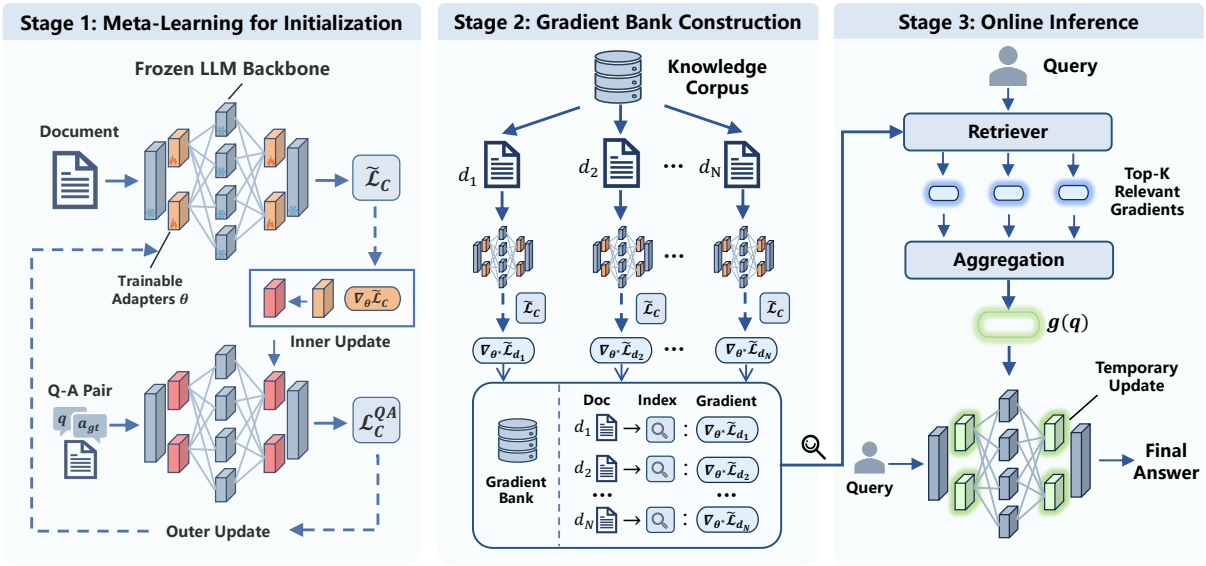


Figure 1: Overview of the REGRAD framework. The method operates in three stages: Meta-Learning, Gradient Bank Construction, and Online Inference.

$$g_C = \nabla_\theta \tilde{\mathcal{L}}_C(\theta), \qquad \theta_C = \theta - \alpha \odot g_C, \tag{2}$$

where α is a set of meta-learned *scalar* step sizes (one scalar per trainable LoRA tensor), and $\odot$ broadcasts each scalar to scale the corresponding gradient tensor. Importantly, $\tilde{\mathcal{L}}_C$ is QA label-free and depends only on C, which later allows us to compute the gradient for every document in the corpus. However, a raw gradient from Eq. (1) mainly optimizes surface-form reconstruction, since next-token prediction rewards verbatim continuation (Allen-Zhu and Li, 2024; Su et al., 2025e). Stage 1 therefore meta-learns θ and α so that the same label-free gradient step becomes useful for downstream QA rather than merely reconstructing the document text.

3.2.2 Outer Objective.

While the inner loop operates on individual documents unsupervised, the outer loop evaluates whether these updates collectively enable the model to answer queries. We assume a meta-training set consisting of tuples $(\mathcal{C}, q, a)$. Here, $\mathcal{C} = \{C_k\}_{k=1}^K$ is a set of context passages containing the necessary facts, and (q, a) is a QA pair grounded in $\mathcal{C}$. In a retrieval scenario, a query typically retrieves multiple documents. To simulate this during meta-training and ensure the learned α and θ can handle multi-source knowledge, we aggregate the gradients from multiple retrieved passages. Specifically, we compute the *knowledge atom* g_{C_k} for each passage individually (as in Eq. 2) and sum them to form a unified update direction:

$$g_{\mathcal{C}} = \sum_{k=1}^{K} g_{C_k}, \qquad \theta_{\mathcal{C}} = \theta - \alpha \odot g_{\mathcal{C}}. \tag{3}$$

A critical design constraint in REGRAD is that during the outer pass, the context $\mathcal{C}$ is excluded from the model's input prompt. The model must generate the answer a strictly from the query q, relying solely on the parametric knowledge injected into $\theta_{\mathcal{C}}$. This forces the meta-optimization to treat θ as a "gradient shaper": it must learn to encode factual content into the gradient $g_{\mathcal{C}_q}$ rather than relying on in-context knowledge injection. The outer loss is thus the standard conditional likelihood of the answer:

$$\mathcal{L}_{\mathcal{C}}^{\mathrm{QA}}(\theta_{\mathcal{C}}) = -\frac{1}{L} \sum_{t=1}^{L} \log p_{(W,\theta_{\mathcal{C}})}(a_t \mid q, a_{<t}). \tag{4}$$

3.2.3 Meta-learning Objective.

Finally, we formally define the meta-learning objective. We optimize the initial parameters θ and step sizes α to minimize the outer QA loss after applying the aggregated unsupervised update. The complete bi-level optimization problem is:

$$\min_{\theta,\alpha} \mathbb{E}_{(\mathcal{C},q,a)\sim\mathcal{D}_{\mathrm{meta}}} \left[\mathcal{L}_{\mathcal{C}}^{\mathrm{QA}} \left(\theta - \alpha \odot \sum_{C \in \mathcal{C}} g_C(\theta) \right) \right]. \tag{5}$$

Eq. (5) differentiates through the inner update in Eq. (2), so the meta-gradient propagates through the dependence of the document gradient $g_C(\theta) =$

$\nabla_\theta \tilde{\mathcal{L}}_C(\theta)$ on the initialization θ. Consequently, downstream QA supervision does not directly optimize the model to answer questions; instead, it shapes how unsupervised language-modeling gradients are computed. From an optimization perspective, meta-training encourages each unsupervised document gradient to function as a task-relevant one-step update: although g_C is computed without QA labels, its direction is learned to reduce the QA loss after aggregation and application. In this sense, θ acts as a *gradient shaper*, transforming language-modeling gradients into adaptation directions that are useful for question answering. Note that once meta-training is complete, (θ^*, α^*) are fixed and used to generate the gradient bank in Stage 2.

3.3 Stage 2: Gradient Bank Construction

Given the meta-learned initialization and step sizes (θ^*, α^*), Stage 2 constructs a *Gradient Bank* that converts the entire knowledge corpus into a collection of retrievable gradients. For each document $d_i \in \mathcal{D}$, we compute an unsupervised language-modeling gradient in the plug-in parameter space using the meta-learned initialization: $g_i = \nabla_\theta \tilde{\mathcal{L}}_{d_i}(\theta^*)$. This gradient g_i serves as a compact *parametric knowledge representation* of the factual knowledge in d_i. Crucially, due to the meta-learning stage (Stage 1), these gradients are reshaped to better support downstream question answering rather than merely optimizing surface-form reconstruction. As a result, each g_i can be interpreted as a *knowledge atom* that captures how the model parameters should change in order to internalize the information in d_i.

To enable query-time retrieval, we associate each gradient g_i with its corresponding document text d_i and build a retrieval index over the corpus. The choice of indexing strategy is orthogonal to the ReGrad framework and depends on the downstream retrieval setup: lexical methods (e.g., inverted indices with BM25) index raw document text, while dense retrievers index learned document embeddings. At inference time, retrieval operates solely over document representations, and the retrieved document IDs are then used to fetch their stored gradients.

3.4 Stage 3: Online Inference

At inference time, given a user query q, we retrieve top-K most relevant documents $\{d_{i_k}\}_{k=1}^K$ and fetch their stored knowledge atoms $\{g_{i_k}\}_{k=1}^K$ from the Gradient Bank. We combine retrieved knowledge atoms by plain gradient accumulation:

$$g(q) = \sum_{k=1}^{K} g_{i_k}, \qquad \theta_q = \theta^* - \alpha^* \odot g(q). \tag{6}$$

The answer is generated using $\mathcal{M}_{(W,\theta_q)}$ without appending retrieved passages, and after generation, we revert to θ^*, ensuring that no updates accumulate across queries.

The aggregation mechanism in Eq. (6) follows directly from the additivity of gradients. In standard training, techniques such as Distributed Data Parallelism and Gradient Accumulation combine gradients computed from different samples or micro-batches to obtain the gradient of the corresponding summed or averaged loss. Similarly, since all stored knowledge atoms are computed at the same meta-learned initialization θ^*, summing the gradients of the retrieved documents is equivalent to computing the gradient of their aggregate language-modeling objective at θ^*. Therefore, accumulating the retrieved gradients is equivalent to taking the gradient of the aggregate language-modeling objective with respect to the retrieved documents. The resulting update follows the same additive structure as standard gradient-based training, while the meta-learned step sizes control how the aggregated gradient is applied.

4 Experimental Setup

Baselines. We compare REGRAD with direct generation (no external knowledge provided), parametric knowledge injection (Standard CPT and its variant Instruction CPT (Cheng et al., 2023)), non-parametric knowledge injection, and a retrieval-based parameter update method (Parametric RAG (Su et al., 2025e)). For non-parametric knowledge injection, we include both Standard RAG (Lewis et al., 2020) and a stronger prompt-engineered (PE) variant, denoted as PE-RAG. Standard RAG uses the widely adopted RAG prompt adopted by FLARE (Jiang et al., 2023), DRAGIN (Su et al., 2024d), and Parametric RAG (Su et al., 2025e), while PE-RAG differs only by replacing this default prompt with a stronger RAG-specific prompt. We include PE-RAG to control for the effect of prompt quality in non-parametric retrieval augmentation. Unless otherwise specified, all main experiments and ablation studies use the PE-RAG prompt, except for Standard RAG and Parametric RAG, which follow their original prompt settings to maintain consistency with prior

Model	Method	General			Law			Medical			Overall
		2WQA	CWQ	HQA	CHOLD	LHF	HOUSING	PUBMED	MEDQA	BIOASQ	Avg.
LLaMA-1B	**Direct**	21.27*	31.71*	16.08*	42.1*	41.1*	64.3*	17.3*	6.24*	42.6*	31.41*
	Standard CPT	15.34*	28.19*	11.42*	40.1*	40.3*	59.7*	18.9*	6.13*	46.6*	29.63*
	Instruction CPT	30.71*	34.74*	18.93*	40.7*	47.1*	62.9*	20.1*	6.80*	41.4*	33.71*
	RAG (ICL)	24.08*	33.58*	24.01*	42.7*	41.2*	65.8*	70.2*	7.88*	71.2*	42.29*
	PE-RAG	24.98*	35.63*	27.94*	44.0*	43.2*	62.7*	62.9*	6.12*	66.3*	41.53*
	PRAG	24.73*	32.27*	20.11*	41.8*	41.2*	63.5*	17.3*	6.92*	47.3*	32.79*
	Fine-tuned-Direct	27.96*	42.74*	18.66*	45.2*	86.1*	79.2*	79.0*	10.63	72.7*	51.35*
	Fine-tuned-RAG	32.04*	46.82*	31.09*	42.1*	90.9*	68.4*	78.2*	10.82	80.8	53.46*
	Fine-tuned-PRAG	31.43*	45.26*	24.73*	65.1*	89.5*	81.9	92.4*	10.78	76.8*	57.54*
	ReGrad	34.61*	47.18*	30.72*	65.7	**94.8**	82.1	93.3	10.12	74.6*	59.24*
	ReGrad + ICL	**40.63**	**50.34**	**39.18**	**68.4**	94.7	**84.5**	**97.9**	**11.50**	**83.4**	**63.39**
LLaMA-3B	**Direct**	16.58*	36.69*	18.77*	55.8*	46.1*	47.7*	88.1*	10.63*	77.8*	44.24*
	Standard CPT	10.96*	33.94*	15.48*	54.2*	44.5*	45.7*	78.4*	9.47*	75.1*	40.86*
	Instruction CPT	30.28*	43.26*	24.09*	55.5*	48.2*	45.4*	82.6*	13.06*	78.3*	46.74*
	RAG (ICL)	20.13*	29.84*	24.11*	63.2*	42.2*	51.2*	82.8*	7.13*	74.2*	43.87*
	PE-RAG	17.75*	40.05*	32.92*	66.2*	47.5*	53.6*	81.7*	11.58*	77.9*	47.69*
	PRAG	19.34*	38.22*	20.73*	56.0*	43.8*	48.1*	89.1*	8.18*	79.4*	44.77*
	Fine-tuned-Direct	29.07*	47.27*	22.88*	45.0*	87.7*	79.0*	79.1*	15.08*	76.1*	53.47*
	Fine-tuned-RAG	35.48*	51.11	32.49*	50.9*	91.9	70.5*	86.4*	15.19*	82.1*	57.34*
	Fine-tuned-PRAG	32.39*	51.42	29.51*	64.2*	89.2*	82.5	92.7	16.12	79.1*	59.68*
	ReGrad	36.82*	52.57	33.43*	68.8	**94.7**	83.1	92.8	16.66	78.6*	61.94*
	ReGrad + ICL	**44.18**	**54.08**	**43.47**	**71.3**	94.6	**84.8**	**97.9**	**18.21**	**85.7**	**66.03**
LLaMA-8B	**Direct**	33.42*	42.73*	26.34*	61.3*	54.1*	46.0*	91.2*	16.46*	81.2*	50.31*
	Standard CPT	25.67*	40.24*	23.51*	58.1*	53.8*	46.2*	90.3*	15.59*	80.2*	48.18*
	Instruction CPT	36.31*	46.58*	27.44*	63.8*	54.0*	49.5*	38.7*	19.24*	60.6*	44.02*
	RAG (ICL)	26.98*	31.47*	35.23*	66.3*	39.9*	53.5*	80.1*	17.88*	80.5*	47.98*
	PE-RAG	29.61*	36.58*	38.70*	69.7*	51.4*	60.2*	80.6*	14.15*	81.0*	51.33*
	PRAG	31.63*	40.96*	27.18*	60.3*	52.0*	46.2*	92.4*	15.93*	82.1*	49.86*
	Fine-tuned-Direct	32.18*	49.67*	28.84*	56.9*	84.2*	80.2*	92.8*	22.89	81.4*	58.79*
	Fine-tuned-RAG	37.17*	54.21	39.16*	59.6*	91.9	69.5*	92.7*	**23.22**	84.6	61.34*
	Fine-tuned-PRAG	35.71*	51.26*	32.97*	66.2*	78.6*	83.0	91.8*	21.37	82.6*	60.39*
	ReGrad	38.43*	56.41	36.31*	70.3*	**96.2**	84.8	93.2*	21.65	81.9*	64.36*
	ReGrad + ICL	**45.06**	**57.52**	**45.27**	**74.1**	94.8	**85.4**	**98.1**	19.83*	**85.3**	**67.26**

Table 1: Main results under the Top-1000 setting on general and domain-specific benchmarks. We compare **ReGrad** with direct generation, CPT variants, RAG, PRAG, and their fine-tuned variants across LLaMA-3.2 1B, 3B, and LLaMA-3.1 8B. The best result in each column and model block is highlighted in **bold**, and the second best is underlined. * denotes a result significantly lower than the bolded result under a t-test ($p < 0.05$). Overall Avg. is the macro average over dataset-level scores, where each dataset uses its task-specific metric.

work. For fairness, we also include fine-tuned variants of applicable baselines. More details on baseline implementations and settings are described in Appendix E.1.

Implementation Details of ReGrad. REGRAD is instantiated on LLaMA-family models across different series and scales, including LLaMA-3.2-1B/3B and LLaMA-3.1-8B. We use BM25 for retrieval using Elasticsearch and aggregate the top-K passages with $K = 3$. For each domain, we meta-train a single LoRA initialization and step-size configuration using the aggregated training splits of the corresponding downstream datasets, while strictly separating meta-training and evaluation data. In addition to the default parametric-only REGRAD setting, we also evaluate REGRAD+ICL, a separately meta-trained variant where the retrieved passages are included in both the outer-loop QA prompt during meta-training and the generation prompt during inference. The bi-level objective for both settings is optimized for 1 epoch with a meta-batch size of 16, a peak outer learning rate of 4×10^{-4}, and bfloat16 precision. Further details, including hyperparameters and prompt templates, are provided in Appendix E.2.

Benchmarks and Corpora. We evaluate ReGrad on both a general-domain Wikipedia setting and two vertical domains (Medicine and Law). For the general domain, we use the DPR Wikipedia dump (Karpukhin et al., 2020) as the external corpus and evaluate on 2WQA (Ho et al., 2020), HotpotQA (Yang et al., 2018), and CWQ (Talmor and Berant, 2018). For vertical domains, we use PubMed Abstracts (The Pile) (Gao et al., 2020) and Pile-of-Law (Henderson et al., 2022) as the corpus, and evaluate on PubMedQA (Jin et al., 2019), MedQA (Jin et al., 2021), and BioASQ (Tsatsaronis et al., 2015), CaseHOLD (Zheng et al., 2021), Learned Hands Family (Guha et al., 2023), and HousingQA (Zheng et al., 2025). More details on

# Docs	2Wiki		CWQ		HotpotQA		Avg.	
	CPT	RG	CPT	RG	CPT	RG	CPT	RG
2.5K	27.14	33.95	43.32	55.91	21.73	23.26	30.73	37.71
5K	28.26	34.12	44.55	57.16	21.43	27.65	31.41	39.64
7.5K	27.43	36.12	45.72	59.84	22.46	31.84	31.87	42.60
10K	20.25	37.70	45.89	60.10	17.26	34.35	27.80	44.05

Table 2: Capability preservation under continual knowledge injection on LLaMA-8B. We report F1 on three benchmarks. RG denotes REGRAD.

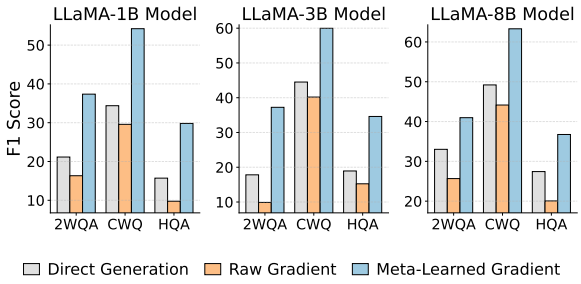


Figure 2: Impact of the meta-learning stage.

datasets and corpora are provided in Appendix E.3.

Evaluation Protocols and Metrics. We use Accuracy for closed-ended tasks and token-level F1 for open-ended QA. All results use greedy decoding with official HuggingFace chat templates. The main results are reported for the first 1,000 test instances of each dataset, with other experiments reported for the first 300. More details on evaluation and metrics are provided in Appendix E.4.

5 Experimental Results

5.1 Main Results

Table 1 compares REGRAD with parametric updating baselines (CPT and PRAG) and non-parametric retrieval baselines (RAG). We evaluate REGRAD in two modes: a parameter-only mode using only retrieved gradients, and a hybrid mode, REGRAD + *ICL*, which additionally places the retrieved documents in the input context. In the parameter-only mode, REGRAD achieves the best average performance among non-hybrid methods across all three model scales, showing that document-induced gradients provide an effective reversible adaptation signal. While REGRAD does not uniformly outperform the fine-tuned RAG baseline on every individual benchmark, its strong overall performance demonstrates the value of retrieved gradients alone. The hybrid REGRAD + *ICL* setting achieves the best average performance across all model scales and the strongest results on most benchmarks. These results show that retrieved gradients complement in-context evidence, positioning REGRAD as either a standalone parameter-only adaptation method or a parameter-side module for retrieval-augmented generation.

5.2 Cumulative Drift Analysis

We further examine whether REGRAD avoids the cumulative drift observed in continual post-training as knowledge injection scales. On LLaMA-3.1-8B, we build a 10K-document corpus by combining the top-3 retrieved documents for all downstream queries with randomly sampled noise documents, and incrementally inject the shuffled corpus into the model. CPT uses the same LoRA architecture and all applicable hyperparameters as REGRAD. The difference is that CPT continuously updates model parameters, while REGRAD stores document-induced gradients and applies only retrieved gradients as temporary query-specific updates. As shown in Table 2, CPT initially improves performance but eventually degrades as more documents are injected, suggesting the effect of cumulative parameter drift. In contrast, REGRAD consistently improves across all datasets as the Gradient Bank grows, because a larger bank increases the likelihood of retrieving relevant gradients while each query still applies only a one-step, temporary update. The average trend further confirms that REGRAD scales knowledge injection without the long-term degradation observed in CPT.

5.3 Ablation Studies

Necessity of Meta-Learning for Gradient Alignment. To validate the effectiveness of our bi-level optimization, we compare ReGrad against the Raw Gradient baseline, where gradients are derived directly from the standard next-token prediction objective without meta-training. As illustrated in Figure 2, the Raw Gradient approach consistently underperforms Direct Generation across all model scales and benchmarks. This performance degradation highlights a fundamental misalignment: raw gradients are optimized for surface-form reconstruction (verbatim memorization), which, when applied naively, introduces optimization noise that disrupts the model's pre-trained capabilities. This result confirms that effective knowledge injection cannot be achieved by simply caching optimization signals. ReGrad's meta-learning is therefore indispensable, successfully transforming gradients from disruptive reconstruction signals into aligned adaptation artifacts that enhance downstream tasks.

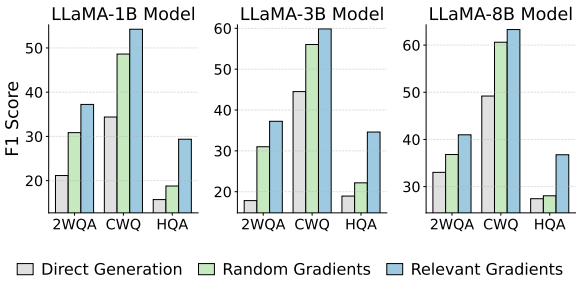


Figure 3: Impact of gradient relevance.

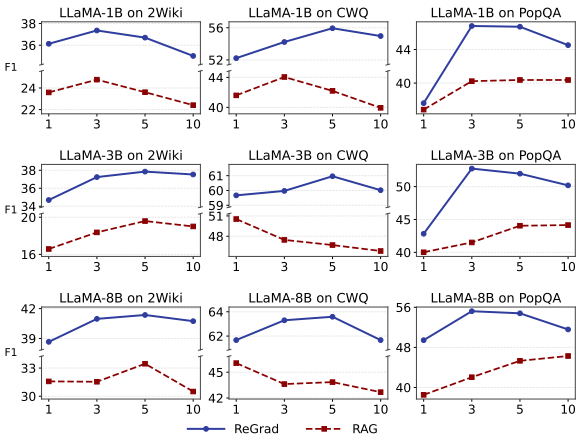


Figure 4: Impact of the number of injected gradients.

Impact of Gradient Relevance. To investigate whether the performance gains stem from specific knowledge injection or merely a meta-learned task adaptation, we conduct an ablation study by replacing the retrieved gradients with randomly sampled gradients from the Gradient Bank (referred to as "Random Gradients"). As illustrated in Figure 3, applying random gradients yields only a marginal improvement over the direct generation baseline, while injecting relevant gradients remains significantly superior. This observation implies that the gradient signal carries both a generic task-solving schematic and specific factual knowledge. Random gradients only trigger a general "QA mode," explaining the marginal performance gain. Crucially, the significant gain from relevant gradients confirms that specific semantic information is the key performance driver, validating that stored gradients effectively encode unique document knowledge into the LLM's parametric space.

5.4 Effect of Injected Gradient Number

We study how the number of injected gradients affects REGRAD by varying the number of retrieved documents $k \in \{1, 3, 5, 10\}$. As shown in Figure 4, REGRAD consistently outperforms RAG in absolute performance, while both methods exhibit an inverted-U trend. Increasing k from 1 to 3 improves recall and mitigates retriever failure when the gold document is not ranked first. However, further increasing k introduces different failure modes: RAG may suffer from lost-in-the-middle effects and attention dilution, whereas REGRAD may suffer from *gradient interference*, where irrelevant gradients harm the temporary update. The trend is also task-dependent: REGRAD is more robust on complex reasoning tasks such as CWQ, while RAG scales better on simple entity-centric tasks such as PopQA. To further isolate the effect of noisy retrieval, we report an additional stress test with fixed relevant passages and random distractors in Appendix B.

Model	Method	General	Med	Law
1B	**Base LLM**	22.06	27.50	45.33
	General-init ReGrad	42.05	28.12	43.33
	Continued ReGrad	38.42	56.07	79.11
	Mixed-domain ReGrad	42.45	55.35	76.44
3B	**Base LLM**	25.36	59.72	48.22
	General-init ReGrad	46.14	45.46	53.67
	Continued ReGrad	43.18	61.72	79.44
	Mixed-domain ReGrad	47.80	60.70	76.55

Table 3: Cross-domain transfer of REGRAD. General-init applies a general-domain meta-initialization to all domains, while Continued and Mixed-domain further adapt it for domain transfer.

5.5 Cross-Domain Transfer

We further examine whether a general-domain meta-initialization can support REGRAD across domains. In Table 3, Base LLM denotes direct generation without any augmentation. General-init REGRAD uses the meta-initialization learned from the general domain to construct Gradient Banks for all domains. Although this setting performs well on the General domain, it brings limited or even negative gains on Medical and Law tasks, indicating that the meta-initialization is domain-dependent. We then evaluate two simple transfer strategies: *Continued ReGrad*, which further adapts the general-domain initialization on target-domain data, and *Mixed-domain ReGrad*, which trains the initialization with both general and target-domain data. Both strategies substantially improve Medical and Law performance while preserving strong General performance. These results suggest that REGRAD can be transferred to specialized domains, but its meta-initialization requires adaptation.

6 Efficiency Analysis

We briefly discuss the efficiency of REGRAD; detailed discussion and complexity analysis are provided in Appendix C.

Offline computation. REGRAD shifts adaptation cost to the offline construction of the Gradient Bank. For each document, it performs a single backward pass restricted to the low-rank plug-in parameters, making the total construction cost comparable to one epoch of CPT in the LoRA subspace. In our measurements on an NVIDIA A100 GPU, gradient construction takes 50.97, 71.17, and 84.05 ms/doc for the 1B, 3B, and 8B models, respectively.

Online inference. Unlike RAG, which lengthens the input with retrieved documents, REGRAD injects knowledge through lightweight low-rank updates without adding tokens. Applying retrieved gradients requires only tensor additions, incurring just 6-15 ms across the 1B, 3B, and 8B models. Importantly, REGRAD supports batched inference despite using query-specific retrieved gradients. This is analogous to multi-LoRA serving: requests share the same backbone while applying different LoRAs, a setting that systems such as Punica (Chen et al., 2024) and S-LoRA (Sheng et al., 2023) have shown can be efficiently batched in practice. Appendix D reports measured end-to-end latency across baselines and model scales.

Storage cost. The main trade-off of REGRAD is storage, as the Gradient Bank scales linearly with the corpus size. In our 1B setting, each document gradient requires a few hundred KB under the low-rank plug-in parameterization (detailed analysis is in Appendix C). This cost can be further reduced by caching gradients only for frequently accessed or high-value documents, while computing gradients for long-tail documents on demand.

7 Conclusion

In this paper, we introduced REGRAD, a paradigm that treats document-induced gradients as retrievable knowledge units and optimizes them through a bi-level meta-learning objective. By retrieving and temporarily applying query-relevant gradients, REGRAD enables selective parametric knowledge injection without permanent weight drift. Experiments show that REGRAD outperforms CPT and RAG baselines while remaining complementary to in-context RAG.

Acknowledgements

This work is supported by the Research Project of Quancheng Laboratory, China (Grant No. QCL20250105)

Limitations

As an initial step toward the broader paradigm of retrievable parametric adaptation, this work demonstrates the feasibility of treating gradients as retrievable knowledge units. Several limitations remain and point to important directions for future research. First, the current ReGrad framework relies on task-specific supervised data during the meta-learning stage to align document-derived gradients with downstream knowledge use. Future work could explore weakly supervised or synthetic-data-based meta-training to reduce this dependence. Second, ReGrad requires storing document-specific gradients in a Gradient Bank. Although this follows the same general assumption as Parametric RAG methods that store retrievable parametric artifacts, it introduces additional storage overhead compared with text-only retrieval. More compact gradient representations, sparsification, quantization, or selective storage strategies could further improve scalability. Finally, our experiments focus on injecting factual knowledge for knowledge-intensive tasks. However, gradients may also encode reusable capabilities rather than only document-level knowledge. Extending ReGrad to capability injection, such as retrieving and applying gradients associated with agent skills or tool-use behaviors, is an important direction for future research.

References

Zeyuan Allen-Zhu and Yuanzhi Li. 2024. Physics of language models: Part 3.1, knowledge storage and extraction. In *Forty-first International Conference on Machine Learning*.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. 2022. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR.

Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2024. Punica: Multi-tenant lora serving. *Proceedings of Machine Learning and Systems*, 6:1–13.

Zeming Chen, Gail Weiss, Eric Mitchell, Asli Celikyilmaz, and Antoine Bosselut. 2023. Reckoning: Reasoning through dynamic knowledge encoding. *Advances in Neural Information Processing Systems*, 36:62579–62600.

Daixuan Cheng, Shaohan Huang, and Furu Wei. 2023. Adapting large language models via reading comprehension. In *The Twelfth International Conference on Learning Representations*.

Qian Dong, Qingyao Ai, Hongning Wang, Yiding Liu, Haitao Li, Weihang Su, Yiqun Liu, Tat-Seng Chua, and Shaoping Ma. 2025. Decoupling knowledge and context: An efficient and effective retrieval augmented generation framework via cross attention. In *Proceedings of the ACM on Web Conference 2025*, pages 4386–4395.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Yan Fang, Jingtao Zhan, Qingyao Ai, Jiaxin Mao, Weihang Su, Jia Chen, and Yiqun Liu. 2024. Scaling laws for dense retrieval. *arXiv preprint arXiv:2403.18684*.

Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, pages 1126–1135. PMLR.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. *arXiv e-prints*, page arXiv:2101.00027.

Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. 2023. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in neural information processing systems*, 36:44123–44279.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. 2020a. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 8342–8360.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A Smith. 2020b. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. 2020. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR.

Moritz Hardt and Yu Sun. 2024. Test-time training on nearest neighbors for large language models. In *The Twelfth International Conference on Learning Representations*.

Peter Henderson, Mark S. Krass, Lucia Zheng, Neel Guha, Christopher D. Manning, Dan Jurafsky, and Daniel E. Ho. 2022. Pile of law: Learning responsible data filtering from the law and a 256gb open-source legal dataset. In *Advances in Neural Information Processing Systems*.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*.

Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Joel Jang, Seonghyeon Ye, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, Stanley Jungkyu Choi, and Minjoon Seo. 2021. Towards continual knowledge learning of language models. *arXiv preprint arXiv:2110.03215*.

Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421.

Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. 2019. Pubmedqa: A dataset for biomedical research question answering. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 2567–2577.

Ishan Jindal, Chandana Badrinath, Pranjal Bharti, Lakkidi Vinay, and Sachin Dev Sharma. 2024. Balancing continuous pre-training and instruction fine-tuning: Optimizing instruction-following in llms. *arXiv preprint arXiv:2410.10739*.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.

Angeliki Lazaridou, Adhi Kuncoro, Elena Gribovskaya, Devang Agrawal, Adam Liska, Tayfun Terzi, Mai Gimenez, Cyprien de Masson d'Autume, Tomas Kocisky, Sebastian Ruder, et al. 2021. Mind the gap: Assessing temporal generalization in neural language models. *Advances in Neural Information Processing Systems*, 34:29348–29363.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

Zhizhong Li and Derek Hoiem. 2017. Learning without forgetting. *IEEE transactions on pattern analysis and machine intelligence*, 40(12):2935–2947.

Yong Lin, Hangyu Lin, Wei Xiong, Shizhe Diao, Jianmeng Liu, Jipeng Zhang, Rui Pan, Haoxiang Wang, Wenbin Hu, Hanning Zhang, et al. 2024. Mitigating the alignment tax of rlhf. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 580–606.

David Lopez-Paz and Marc'Aurelio Ranzato. 2017. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30.

Wei Lu, Rachel K Luu, and Markus J Buehler. 2025. Fine-tuning large language models for domain adaptation: Exploration of training strategies, scaling, model merging and synergistic capabilities. *npj Computational Materials*, 11(1):84.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022a. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022b. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D Manning. 2021. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*.

Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D Manning, and Chelsea Finn. 2022. Memory-based model editing at scale. In *International Conference on Machine Learning*, pages 15817–15831. PMLR.

Neel Nanda, Senthooran Rajamanoharan, János Kramár, and Rohin Shah. 2023. Fact finding: Attempting to reverse-engineer factual recall on the neuron level. Accessed: 2025-01-24.

Fabio Petroni, Tim Rocktäschel, Sebastian Riedel, Patrick Lewis, Anton Bakhtin, Yuxiang Wu, and Alexander Miller. 2019. Language models as knowledge bases? In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 2463–2473.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *arXiv preprint arXiv:2302.00083*.

Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.

David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *nature*, 323(6088):533–536.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, et al. 2023. S-lora: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*.

Weihang Su, Qingyao Ai, Xiangsheng Li, Jia Chen, Yiqun Liu, Xiaolong Wu, and Shengluan Hou. 2024a. Wikiformer: Pre-training with structured information of wikipedia for ad-hoc retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19026–19034.

Weihang Su, Qingyao Ai, Yueyue Wu, Yixiao Ma, Haitao Li, and Yiqun Liu. 2023. Caseformer: Pre-training for legal case retrieval. *arXiv preprint arXiv:2311.00333*.

Weihang Su, Qingyao Ai, Yueyue Wu, Anzhe Xie, Changyue Wang, Yixiao Ma, Haitao Li, Zhijing Wu, Yiqun Liu, and Min Zhang. 2025a. Pre-training for legal case retrieval based on inter-case distinctions. *ACM Transactions on Information Systems*, 43(5):1–27.

Weihang Su, Qingyao Ai, Jingtao Zhan, Qian Dong, and Yiqun Liu. 2025b. Dynamic and parametric retrieval-augmented generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 4118–4121.

Weihang Su, Xuanyi Chen, Yueyue Wu, Qingyao Ai, and Yiqun Liu. 2026a. Enhancing judgment document generation via agentic legal information collection and rubric-guided optimization. *arXiv preprint arXiv:2605.02011*.

Weihang Su, Qian Dong, Qingyao Ai, and Yiqun Liu. 2025c. Sigir-ap 2025 tutorial proposal: Dynamic and parametric retrieval-augmented generation. In *3rd International ACM SIGIR Conference on Information Retrieval in the Asia Pacific*.

Weihang Su, Yiran Hu, Anzhe Xie, Qingyao Ai, Zibing Que, Ning Zheng, Yun Liu, Weixing Shen, and Yiqun Liu. 2024b. Stard: A chinese statute retrieval dataset with real queries issued by non-professionals. *arXiv preprint arXiv:2406.15313*.

Weihang Su, Jianming Long, Qingyao Ai, Yichen Tang, Changyue Wang, Yiteng Tu, and Yiqun Liu. 2026b. Skill retrieval augmentation for agentic ai. *arXiv preprint arXiv:2604.24594*.

Weihang Su, Jianming Long, Changyue Wang, Shiyu Lin, Jingyan Xu, Ziyi Ye, Qingyao Ai, and Yiqun Liu. 2025d. Towards unification of hallucination detection and fact verification for large language models. *arXiv preprint arXiv:2512.02772*.

Weihang Su, Yichen Tang, Qingyao Ai, Changyue Wang, Zhijing Wu, and Yiqun Liu. 2024c. Mitigating entity-level hallucination in large language models. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, pages 23–31.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024d. Dragin: Dynamic retrieval augmented generation based on the real-time information needs of large language models. *arXiv preprint arXiv:2403.10081*.

Weihang Su, Yichen Tang, Qingyao Ai, Junxi Yan, Changyue Wang, Hongning Wang, Ziyi Ye, Yujia Zhou, and Yiqun Liu. 2025e. Parametric retrieval augmented generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1240–1250.

Weihang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. 2024e. Unsupervised real-time hallucination detection based on the internal states of large language models. *arXiv preprint arXiv:2403.06448*.

Weihang Su, Anzhe Xie, Qingyao Ai, Jianming Long, Xuanyi Chen, Jiaxin Mao, Ziyi Ye, and Yiqun Liu. 2025f. Surge: A benchmark and evaluation framework for scientific survey generation. *arXiv preprint arXiv:2508.15658*.

Weihang Su, Baoqing Yue, Qingyao Ai, Yiran Hu, Jiaqi Li, Changyue Wang, Kaiyuan Zhang, Yueyue Wu, and Yiqun Liu. 2025g. Judge: Benchmarking judgment document generation for chinese legal system. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3573–3583.

Weihang Su, Hanwen Zhang, Qingyao Ai, and Yiqun Liu. 2026c. Decoupling knowledge and task subspaces for composable parametric retrieval augmented generation. *arXiv preprint arXiv:2604.26768*.

Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 641–651, New Orleans, Louisiana. Association for Computational Linguistics.

Yuqiao Tan, Shizhu He, Huanxuan Liao, Jun Zhao, and Kang Liu. 2025. Dynamic parametric retrieval augmented generation for test-time knowledge enhancement. *arXiv preprint arXiv:2503.23895*.

George Tsatsaronis et al. 2015. An overview of the bioasq large-scale biomedical semantic indexing and question answering competition. *BMC Bioinformatics*, 16(1):138.

Yiteng Tu, Weihang Su, Yujia Zhou, Yiqun Liu, and Qingyao Ai. 2025. Robust fine-tuning for retrieval augmented generation against retrieval defects. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1272–1282.

Changyue Wang, Weihang Su, Qingyao Ai, and Yiqun Liu. 2026a. Joint evaluation of answer and reasoning consistency for hallucination detection in large reasoning models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33377–33385.

Changyue Wang, Weihang Su, Qingyao Ai, Yichen Tang, and Yiqun Liu. 2025a. Knowledge editing through chain-of-thought. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 10684–10704.

Changyue Wang, Weihang Su, Qingyao Ai, Yichen Tang, Runzhong Qiao, Xuancheng Li, Min Zhang, and Yiqun Liu. 2026b. Adaptive multi-resolution procedural knowledge compression for large language models. *arXiv preprint arXiv:2606.12203*.

Changyue Wang, Weihang Su, Qingyao Ai, Yujia Zhou, and Yiqun Liu. 2025b. Decoupling reasoning and

knowledge injection for in-context knowledge editing. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24543–24562.

Changyue Wang, Weihang Su, Qingyao Ai, Yujia Zhou, and Yiqun Liu. 2025c. Decoupling reasoning and knowledge injection for in-context knowledge editing. *arXiv preprint arXiv:2506.00536*.

Changyue Wang, Weihang Su, Hu Yiran, Qingyao Ai, Yueyue Wu, Cheng Luo, Yiqun Liu, Min Zhang, and Shaoping Ma. 2024. Lekube: A legal knowledge update benchmark. *arXiv preprint arXiv:2407.14192*.

Xingjin Wang, Howe Tissue, Lu Wang, Linjing Li, and Daniel Dajun Zeng. 2025d. Learning dynamics in continual pre-training for large language models. *arXiv preprint arXiv:2505.07796*.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. 2024. Less: selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*, pages 54104–54132.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*.

Zeping Yu and Sophia Ananiadou. 2024. Neuron-level knowledge attribution in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 3267–3280, Miami, Florida, USA. Association for Computational Linguistics.

Lucia Zheng, Neel Guha, Brandon R Anderson, Peter Henderson, and Daniel E Ho. 2021. When does pretraining help? assessing self-supervised learning for law and the casehold dataset of 53,000+ legal holdings. In *Proceedings of the eighteenth international conference on artificial intelligence and law*, pages 159–168.

Lucia Zheng, Neel Guha, Javokhir Arifov, Sarah Zhang, Michal Skreta, Christopher D. Manning, Peter Henderson, and Daniel E. Ho. 2025. A Reasoning-Focused Legal Retrieval Benchmark. *arXiv e-prints*, page arXiv:2505.03970.

A Additional Discussion of Related Work

A.1 Continual Post-Training and Knowledge Editing.

Continued post-training and fine-tuning are standard paradigms for injecting new knowledge into LLMs (Gururangan et al., 2020b). A core challenge in these paradigms is catastrophic forgetting and capability drift under long sequences of updates, motivating the use of mitigation techniques in continual learning, such as regularization, distillation, or replay (Kirkpatrick et al., 2017; Li and Hoiem, 2017; Lopez-Paz and Ranzato, 2017). In parallel, knowledge editing aims to precisely update facts with minimal side effects (Meng et al., 2022a). Representative editors include localized weight updates (Meng et al., 2022b), meta-learned editors that transform gradients into parameter shifts (Mitchell et al., 2021), and memory-based editors (Mitchell et al., 2022).

A.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) has become a widely used paradigm for equipping LLMs with external knowledge without modifying their parameters (Lewis et al., 2020; Dong et al., 2025; Tu et al., 2025; Su et al., 2025b,f). Instead of internalizing new information through additional training, RAG retrieves query-relevant passages at inference time and provides them to the model as part of the input context. This non-parametric design makes RAG particularly attractive for hallucination mitigation (Wang et al., 2026a; Su et al., 2025d, 2024e; Wang et al., 2025b), knowledge updates (Wang et al., 2025c,a, 2024), and domains adaption without full model retraining (Su et al., 2024b, 2025g,a, 2026a, 2023). A large body of work has improved different components of the RAG pipeline, including sparse and dense retrieval (Robertson et al., 2009; Su et al., 2024a; Fang et al., 2024), adaptive retrieval strategies (Jiang et al., 2023; Su et al., 2024d,c), graph-structured retrieval (Edge et al., 2024), parametric RAG (Su et al., 2025e,c, 2026c; Wang et al., 2026b), and agentic retrieval frameworks (Jin et al., 2025; Su et al., 2026b). REGRAD shares the retrieval-based philosophy of RAG, but changes what is retrieved and how the retrieved information is used.

A.3 Test-Time Training.

Recent works explore training the model at inference time, e.g., test-time fine-tuning on retrieved neighbors (Hardt and Sun, 2024; Chen et al., 2023). Such methods demonstrate that a few gradient steps can yield strong specialization, but require online backpropagation and training, which will introduce significant latency. ReGrad bypasses the latency constraints of online backpropagation by retrieving pre-computed, meta-optimized parameter shifts. This approach decouples optimization from inference, thereby retaining the benefits of instance-specific adaptation while incurring negligible run-

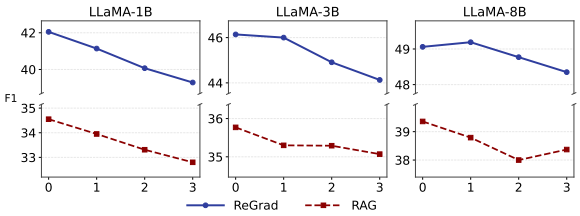


Figure 5: Average performance across four datasets with varying levels of noise (3 relevant documents plus n random documents). REGRAD maintains a consistent performance advantage over RAG across all model scales despite similar degradation trends.

time overhead.

B Additional Robustness Analysis

In Section 5.4, we analyze how REGRAD behaves when merging gradients from an increasing number of retrieved documents. Here, we provide a complementary stress test that explicitly isolates the effect of noisy retrieval. Specifically, we keep a fixed set of $k = 3$ relevant passages and inject $n \in \{0, 1, 2, 3\}$ random distractor documents. As illustrated in Figure 5, REGRAD consistently outperforms RAG across all noise levels and model sizes, demonstrating that our parametric adaptation yields stronger knowledge utilization even in the presence of retrieval noise. However, we also observe that REGRAD exhibits a performance degradation pattern parallel to that of RAG as noise increases. This indicates that while REGRAD effectively capitalizes on relevant information, it remains sensitive to irrelevant signals introduced by noisy retrieval. This finding points to important directions for future optimization: enhancing robustness not only through upstream retrieval denoising, but also by exploring more selective mechanisms for merging knowledge atoms beyond the current accumulation strategy.

C Detailed Discussion on Time and Space Efficiency

We analyze the computational and storage cost of REGRAD, with a particular focus on how it compares with continual post-training and RAG.

C.1 Computation Cost

Meta-training overhead. The meta-learning stage optimizes the initialization θ and step sizes α such that unsupervised document gradients become useful for downstream QA. In practice, this stage incurs a relatively small one-time cost because the meta-objective converges on only a few thousand training instances, and the total computation is negligible compared to corpus-scale processing. We therefore focus our discussion on the dominant cost: gradient construction over the knowledge corpus.

Offline gradient construction. For each document $d_i \in \mathcal{D}$, REGRAD computes a single backward pass of the language modeling loss restricted to the low-rank plug-in parameters θ. Crucially, this gradient is computed *once* and stored in the Gradient Bank. As a result, the total offline cost is equivalent to performing one epoch of continual post-training over the corpus, except that updates are confined to a small LoRA subspace rather than the full model. Compared to CPT, which typically requires multiple epochs to integrate corpus-level knowledge, REGRAD reduces the total training computation by a factor proportional to the number of CPT epochs. If CPT performs n epochs over $\mathcal{D}$, then REGRAD incurs approximately $\frac{1}{n}$ of the total optimization cost, while avoiding cumulative parameter drift.

Online inference cost. At inference time, REGRAD retrieves a small set of gradients $\{g_i\}_{i=1}^{t}$ and aggregates them into a temporary update applied to the plug-in parameters. This operation consists of simple tensor additions and scalar scaling in the low-rank space and is computationally negligible compared to even a single token-level forward pass.

Let $|q|$ denote the query length and h the hidden dimension. The inference-time complexity of REGRAD remains

$$\mathcal{O}(|q|^2 h + |q|h^2), \tag{7}$$

identical to that of the base LLM. In contrast, in-context RAG expands the input by concatenating retrieved documents, yielding a complexity of

$$\mathcal{O}\big((|q| + t|d|)^2 h + (|q| + t|d|)h^2\big), \tag{8}$$

which grows quadratically with the total retrieved context length. Therefore, REGRAD strictly outperforms in-context RAG in online efficiency: it achieves query-conditioned knowledge activation without increasing context length. The trade-off is explicit: REGRAD shifts computation from the online phase to an offline, amortizable preprocessing stage.

When is REGRAD more compute-efficient? Consider a corpus of 100k documents, each containing $|d|$ tokens, and an RAG system that retrieves $t = 3$ documents per query. We quantify computational cost in units of token-wise forward passes. In this analysis, we omit the costs of RAG indexing and REGRAD's Meta-Learning Initialization phase, as they are negligible compared to the construction of the Gradient Bank. Additionally, since REGRAD stores LoRA gradients, the computation required to merge or insert gradients is significantly lower than a single token forward pass through the full LLM and is therefore disregarded.

Based on these assumptions, we now perform a quantitative comparison of the computational overheads. Constructing the Gradient Bank requires 100k backward passes; assuming the computational cost of a backward pass is approximately three times that of a forward pass, the total offline precomputation is equivalent to forward-propagating $300\text{k} \cdot |d|$ tokens. In contrast, RAG incurs an additional forward computation overhead proportional to $3|d|$ tokens for *every* query. Consequently, once the volume of served queries exceeds the corpus size (i.e., 100k queries), the amortized cost of REGRAD becomes lower than that of RAG. This regime is characteristic of long-lived knowledge bases or high-traffic QA systems, where offline preprocessing costs are effectively amortized across a large number of queries.

C.2 Storage Overhead

The storage cost of REGRAD arises from storing per-document gradients in the plug-in parameter space. Let r denote the LoRA rank, n the number of Transformer layers, h the hidden dimension, and l the FFN intermediate dimension. Each document gradient contains $2nr(h + l)$ parameters. While this is larger than storing raw text alone, the representation is compact relative to full-model checkpoints and scales linearly with the number of documents. In practice, this storage overhead is manageable for several reasons. First, access patterns in real-world systems are highly skewed: a small fraction of documents accounts for the majority of queries. Maintaining gradient representations only for high-frequency or high-value documents already captures most of the benefit. Furthermore, when storage is constrained, gradients need not be permanently stored. They can be computed on the fly at inference time using the fixed (θ^*, α^*). As shown in our experiments, online gradient computation introduces only millisecond-level latency, making it feasible for interactive settings. This flexibility allows practitioners to trade storage for computation depending on system constraints.

Table 4: End-to-end inference latency in milliseconds. REGRAD Cached loads pre-computed gradients from the Gradient Bank, while REGRAD Online computes document gradients on the fly without storing cached gradients.

Method	1B	3B	8B
Direct Generation	13.44 ms	18.36 ms	28.30 ms
RAG	17.03 ms	23.62 ms	34.63 ms
REGRAD Cached	16.89 ms	22.97 ms	30.84 ms
REGRAD + ICL	18.21 ms	26.60 ms	38.10 ms
REGRAD Online	32.04 ms	45.83 ms	56.31 ms

D End-to-End Latency

We further conduct an end-to-end latency evaluation to quantify the practical cost of dynamic low-rank update merging and to examine the storage-latency trade-off of REGRAD. All experiments are conducted on an 8×A100-80G server. We report the average latency over 10 repeated runs on 100 randomly sampled general-domain questions. Table 4 shows that the additional serving overhead of cached REGRAD is small. For the 8B model, Direct Generation takes 28.30 ms, while REGRAD Cached takes 30.84 ms. This confirms that dynamic low-rank update merging introduces negligible latency overhead in practice. We also evaluate REGRAD Online, an extreme setting that does not store a cached Gradient Bank. Instead, the document gradients are computed on the fly at inference time. This removes the storage cost of cached gradients, while retaining the raw document corpus and retrieval index. As expected, this setting is slower than cached REGRAD, but the overhead remains moderate. For the 8B model, REGRAD Online takes 56.31 ms, which is 25.47 ms higher than cached REGRAD. This result suggests a practical hybrid deployment strategy: systems can cache gradients for frequently retrieved head documents to maximize inference speed, while computing gradients for long-tail documents on demand to reduce storage usage.

Overall, these results complement the theoretical efficiency analysis by showing that REGRAD supports a flexible latency–storage trade-off. Cached gradients provide near-RAG-level latency with negligible dynamic merging cost, whereas on-the-fly

gradient computation offers a storage-saving alternative when full Gradient Bank caching is undesirable.

E Experimental Setup

In this section, we detail the experimental settings used to evaluate our proposed ReGrad framework. We begin by introducing our selected benchmark datasets and evaluation metrics (§E.3 and §E.4). Next, we introduce our selected baseline methods (§E.1) and implementation. Finally, we provide implementation details regarding our meta-learning process, retrieval strategy, and inference settings (§E.2).

E.1 Baselines

We compare against a broad set of baselines, including direct generation (no external knowledge provided), parametric knowledge injection (Standard CPT and its variant Instruction CPT (Cheng et al., 2023)), non-parametric knowledge injection (Standard RAG (Lewis et al., 2020)), and a retrieval-based parameter-update method (Parametric RAG (Su et al., 2025e)). Below, we introduce each baseline and present its implementation details.

- **Direct Generation:** This baseline evaluates the backbone LLM's intrinsic capabilities on downstream tasks without any external knowledge injection. We use the official Hugging Face inference implementation and follow the standard generation configurations provided by the model developers. The specific prompt templates used for zero-shot inference are detailed in Appendix F.

- **Standard Continual Post-Training (Standard CPT):** Representing the conventional domain adaptation approach, we update the model parameters on the external corpus using a standard causal language modeling objective. To avoid diluting this baseline with a large number of irrelevant documents, we construct its training corpus from the top-3 documents retrieved for each question in the training set, rather than using the entire external corpus. We then perform LoRA fine-tuning on the external knowledge corpus for 1 epoch using the standard language modeling objective. We utilize a learning rate of $5e^{-5}$ and a batch size of 16. The inference configuration remains consistent with Direct Generation.

- **Instruction Continual Post-Training (Instruction CPT)** (Cheng et al., 2023): To mitigate the potential loss of reasoning capabilities during domain adaptation, we adopt the core philosophy of this framework, which transforms raw corpora into reading comprehension tasks. However, our implementation diverges from Cheng et al. (2023) in the data construction pipeline. While the original method relies on unsupervised rule matching to extract Question-Answer (QA) pairs, our preliminary experiments indicate that this approach yields suboptimal performance. Consequently, we replace the rule-based heuristic with an LLM-based generation approach, prompting the model to synthesize high-quality QA pairs for each document (templates provided in Appendix F). We then post-train the model using a language modeling objective on the concatenated sequence of the document context and the generated QA pair. The training hyperparameters align with Standard CPT (learning rate $1e^{-5}$, batch size 16, 1 epoch).

- **Standard RAG:** We employ the conventional Retrieval-Augmented Generation framework, which serves as a strong non-parametric baseline. Following the settings in the PRAG library, we retrieve the top-3 relevant documents from the external corpus and prepend them to the input prompt as context. This allows the model to use external information through in-context learning without modifying its weights. The prompt templates are aligned with those used in ReGrad and are listed in Appendix F.

- **Parametric RAG** (Su et al., 2025e): This recent approach bridges the gap between RAG and CPT by injecting relevant documents directly into the LLM's parameters via document parameterization, rather than consuming context window space. We utilize the official implementation from the PRAG library to compare ReGrad against state-of-the-art retrieval-based parameter-update methods.

- **Finetuned Direct Generation:** To ensure a fair comparison with ReGrad's meta-learning phase, we implement a supervised fine-tuning (SFT) baseline. We fine-tune the base LLM on QA training sets that have no overlap with the corresponding test sets, producing separate models for the General, Law, and Medical domains. The model is trained to generate the answer given

the question, with the language modeling loss calculated solely on the answer tokens. We employ LoRA for efficient fine-tuning, setting the learning rate to $4e^{-4}$ for 1 epoch.

- **Finetuned Standard RAG:** We fine-tune the base LLM using the same hyperparameters as Finetuned Direct Generation (LoRA, learning rate $4e^{-4}$ for 1 epoch). Crucially, unlike the Direct Generation baseline, we retrieve the top-3 relevant documents and append them to the input prompt during both training and inference. This aligns the training distribution with the retrieval-augmented inference setting.

- **Finetuned Parametric RAG:** Similar to Finetuned Standard RAG, this baseline fine-tunes the model on downstream QA pairs. However, instead of appending documents to the context window, it utilizes the Parametric RAG mechanism to inject the top-3 retrieved documents into the model parameters during the forward pass. All training hyperparameters remain consistent with those of the other fine-tuned baselines.

To facilitate reproducibility, we have made our full implementation publicly available. **Further details regarding the setting, hyperparameters, code, data, and models can be accessed via our GitHub repository**[3].

E.2 Implementation Details

Meta-Learning Configuration. We optimize the bi-level objective (Eq. 5) for 1 epoch with a meta-batch size of 16. The outer optimization employs a linear learning rate scheduler with a peak learning rate of $4e^{-4}$. Specifically, we implement the gradient aggregation in Eq. 3 using top-K retrieved passages with $K = 3$. To accommodate these inputs, we set the maximum context length to $1,536$ tokens. All meta-training experiments use bfloat16 mixed precision. Regarding initialization, the LoRA parameters θ follow the standard initialization from Hu et al. (2022), while the learnable scalar step sizes α are initialized to $1e^{-2}$, providing a stable starting point for the inner loop updates.

Retrieval and Indexing. While dense retrieval is popular, recent analyses in RAG (Ram et al., 2023) suggest that lexical matching remains highly competitive, particularly for zero-shot generalization. Given its efficiency and proven robustness, we adopt BM25 as our retrieval backbone, implemented via Elasticsearch. This ensures that performance gains in ReGrad are attributed to the gradient injection mechanism rather than the retrieval of superior contexts.

Base Models and Computing Environment. We implement ReGrad using the open-source LLaMA-3.2 family to verify scalability and robustness across varying model sizes. Specifically, we conduct experiments on LLaMA-3.2-1B-Instruct, LLaMA-3.2-3B-Instruct, and LLaMA-3.1-8B-Instruct. All training and inference processes are executed using PyTorch on NVIDIA A100 GPUs (80GB).

Inference Configuration. All evaluations utilize the official Hugging Face implementations of the LLaMA models. We adhere to the default hyperparameters and chat templates provided by the library. To strictly ensure the reproducibility of our reported metrics, we employ greedy decoding (temperature $= 0$) for all generation tasks.

E.3 Benchmarks and Corpus

General Domain. Consistent with standard practice in retrieval-augmented generation (Karpukhin et al., 2020; Su et al., 2024d), we utilize the Wikipedia dump provided by DPR (Karpukhin et al., 2020) as our external knowledge corpus. To evaluate reasoning capabilities and knowledge retrieval over this corpus, we select four diverse benchmarks:

- **2WikiMultihopQA (2WQA)** (Ho et al., 2020): Evaluates multi-hop reasoning by requiring the integration of information across fragmented Wikipedia passages.

- **HotpotQA (HQA)** (Yang et al., 2018): A challenging benchmark for multi-hop QA that necessitates reasoning over multiple supporting documents to derive the answer.

- **PopQA (PQA)** (Mallen et al., 2023): Assesses factual knowledge retrieval, specifically targeting the resolution of entity ambiguity and long-tail knowledge.

- **ComplexWebQuestions (CWQ)** (Talmor and Berant, 2018): Requires answering complex, multi-step questions derived from the web, testing the scalability of the retrieval mechanism.

[3]We have open-sourced all the code, data, and models in the following anonymized GitHub link: https://github.com/oneal2000/ReGrad/

Vertical Domains. To verify the effectiveness of ReGrad in specialized fields, we conduct experiments in Medicine and Law. For each domain, we utilize a specific uncopyrighted corpus to construct the Gradient Bank.

Medicine: We utilize the **PubMed Abstracts** subset of The Pile (Gao et al., 2020) as the reference corpus. Evaluation is performed on three datasets:

- **PubMedQA** (Jin et al., 2019): A biomedical QA task derived from PubMed abstracts, requiring the model to deduce research conclusions (Yes/No/Maybe).
- **MedQA** (Jin et al., 2021): Comprises questions from professional medical board exams, assessing complex clinical reasoning.
- **BioASQ** (Tsatsaronis et al., 2015): We utilize the binary (Yes/No) subset of the BioASQ-12b training data, where questions involve diverse biomedical topics including diseases and pharmaceuticals.

Law: We employ the **Pile-of-Law** (Henderson et al., 2022) as the legal reference corpus. Downstream evaluation includes:

- **CaseHOLD** (Zheng et al., 2021): A multiple-choice task (5 options) requiring the identification of the correct holding statement for a given legal citation context.
- **Learned Hands Family (LHF)** (Guha et al., 2023): A binary classification task determining whether a legal query pertains to family law issues (e.g., custody, divorce).
- **Housing QA** (Zheng et al., 2025): A binary QA dataset assessing reasoning capabilities regarding US state-level housing laws.

E.4 Evaluation Protocols and Metrics

To maintain consistency across diverse benchmarks, we evaluate models using the first 1000 instances from each sub-dataset test set. Performance is assessed via task-specific metrics. Accuracy is applied to all closed-ended classification tasks, including multiple-choice questions in CaseHOLD and binary classification in PubMedQA, BioASQ, LHF, and Housing QA; model outputs are mapped to labels using regular expression parsing. For open-ended question-answering tasks, including 2WQA, HQA, PopQA, CWQ, and MedQA, we use F1 Score to measure overlap between generated responses and ground truth. This metric ensures a robust evaluation of generation quality by penalizing partial hallucinations while rewarding precise recall of relevant information.

F Prompt Templates

This section describes the prompt templates used across all the methods and experiment stages (training vs. validation). We offer a comprehensive account of how the context is structured, how it is incorporated into the prompt, and how this design differs between baseline approaches and our proposed methods, as well as across datasets with different question formats.

F.1 Context Construction

For multi-passage datasets such as 2WikiMultihopQA, HotpotQA, PopQA, and ComplexWebQuestions, each example is associated with a set of retrieved passages, which provide the background knowledge to answer a multihop question. These passages are concatenated into a single context string indexed in a fixed sequential order. Formally, the context is constructed as:

Concatenated Context

Passage 0: {Passage 0}
Passage 1: {Passage 1}
...
Passage k-1: {Passage k-1}

The concatenated context is truncated to a fixed maximum length to satisfy the input constraints of the language models. In our implementation, the context is truncated to at most 1536 tokens during training and 2048 tokens during validation and inference. This enables the model to attend to the most relevant context while keeping the input length within allowable limits.

F.2 Prompt Usage Across Methods

The prompt templates mainly differ in whether the retrieved relevant document is included in the context, and are further adapted across datasets with different question formats. We describe below how each method uses the prompt and whether the context is available during answer generation.

RAG, **Fine-tuned-RAG** and **ReGrad + ICL** use prompts with context. In these settings, the relevant background information is directly exposed to the model, which is expected to exploit it during answer generation. **Direct**, **Standard CPT**, **Instruction CPT**, **PRAG**, **Fine-tuned**, **Fine-tuned-PRAG** and **ReGrad** use prompts without context during training and validation, requiring the model

to rely on parametric knowledge or learned plug-in parameters.

F.3 Prompt Templates

We present all our prompt templates in this subsection. Depending on the dataset type, slight variations are applied to open-ended, yes/no, and multiple-choice QA. For yes/no datasets (e.g., MedQA,PubMedQA, BioASQ, LHF, HousingQA), the same templates are used with task instructions adjusted to enforce binary outputs.

Prompt with context for open-ended QA: This template is used for open-ended question answering tasks when contextual passages are provided:

Prompt with context for open-ended QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
CONTEXT begin
{context}
CONTEXT end
You are a helpful AI assistant who is ready to answer user's question according to the CONTEXT.
Your answer should be concise, which means it must be a short phrase or a single word, and sentences are not allowed!

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt with context for yes/no QA: This template is used for binary (yes/no) question answering tasks when contextual passages are provided:

Prompt with context for yes/no QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
CONTEXT begin
{context}
CONTEXT end
You are a helpful AI assistant who is ready to answer user's question according to the CONTEXT.
Your answer should be concise, which means it must be 'yes' or 'no' only!

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt with context for multiple-choice QA: This template is used for multiple-choice question answering tasks when contextual passages are provided:

Prompt with context for multiple-choice QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
CONTEXT begin
{context}
CONTEXT end
Complete the following excerpt from a US court opinion.
Always answer ONLY with the option letter: the final answer is X, where X in {{A, B, C, D, E}}.

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}
Choices:
A. {c0}
B. {c1}
C. {c2}
D. {c3}
E. {c4}

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt without context for open-ended QA: This template is used for open-ended question answering tasks when no contextual passages are exposed to the model:

Prompt without context for open-ended QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
You are a helpful AI assistant who is ready to answer user's question.
Your answer should be concise, which means it must be a short phrase or a single word, and sentences are not allowed!

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt without context for yes/no QA: This template is used for binary (yes/no) question answering tasks when no contextual passages are exposed to the model:

Prompt without context for yes/no QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
You are a helpful AI assistant who is ready to answer user's question.
Answer only 'yes' or 'no'!

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt without context for multiple-choice QA: This template is used for multiple-choice question answering tasks when no contextual passages are exposed to the model:

Prompt without context for multiple-choice QA

<|begin_of_text|><|start_header_id|>
system<|end_header_id|>
Complete the following excerpt from a US court opinion.
Answer only with the option letter: A, B, C, D or E.

<|eot_id|><|start_header_id|>user<|end_header_id|>
{question}
Choices:
A. {c0}
B. {c1}
.....

<|eot_id|><|start_header_id|>
assistant<|end_header_id|>
Answer:

Prompt for synthetic QA pairs: This template is used for the Instruction CPT baseline, designed to instruct the LLM to generate synthetic QA pairs for each document in the corpus.

Prompt for synthetic QA pairs

I will provide a passage of text, and you need to generate three different questions based on the content of this passage. Each question should be answerable using the information provided in the passage. Additionally, please provide an appropriate answer for each question derived from the passage.
You need to generate the question and answer in the following format:

```
[
    {
        "question": "What is the capital of France?",
        "answer": "Paris"
        "full_answer": "The capital of France is Paris."
    },
]
```

This list should have at least three elements. You only need to output this list in the above format.
Passage:
{passage}

F.4 Prompt Instantiation

For each example, the placeholders `{context}`, `{question}` are replaced with the actual content. Specially for multiple-choice datasets, placeholders `{c0}`–`{c4}` are replaced with the corresponding candidate options. For example:

- In the `{context}` slot, insert the truncated, concatenated passages relevant to the question.
- In the `{question}` slot, insert the input question from the dataset.
- In the `{c0}`–`{c4}` slot, insert the choices relevant to the question.

This unified prompt design ensures that all methods are evaluated under consistent formatting conditions, enabling a fair and controlled comparison across baselines and proposed approaches.